

IMMC 2022 Winter—Problem D

Team Control Number: 22761451

I. Summary

Recently, metaverse has been a hot topic and direction of technology development that is highly valued by the industry. Its actual meaning is vague, but mostly it is recognized as a free space that can allow players to use virtual items and build amazing structures using their imagination. Examples of such include *Minecraft*, *Second World*, and *Roblox*, each being some popular games where structures of all kinds can be created. An important feature of all metaverse is the virtual items, which is a set of building blocks, that can allow players to apply their creativity in building such a world by using simple operations. From the topic of metaverse, we started a project to design a set of building blocks that can create a virtual city Hong Kong through player's creativity and the set's diversity, and also to estimate the computing resources and storage resources needed.

Developing such a set of building blocks that uses simple operations needs to consider the complexity and the granularity of the building blocks. The system cannot be too complex nor too simple. The dimensions of the building blocks cannot be too large or too small. **Our team has decided to design the set of building blocks through two essential elements: the entity, or the shape of the block, and texture.** In this way all of the normal building blocks like windows, doors, and roads, can be created by combining entities and applying the texture upon it. A wide variety of the objects nowadays can be created through the player's imagination of using such blocks. The system is not too complex and can allow players to build freely.

Estimating the storage resources requires us to calculate the object list, entities, and textures. Entities contains its dimensions and flags; texture contains its RGB values and transparency values; the object structure contains its coordinate, employed entity, and texture. Calculating the storage of object, entity, and texture will result in the overall storage. **Overall, the estimated storage resources of the virtual city Hong Kong is about 6841 GB of storage.**

Estimating the computing resources requires us to calculate the collision, light and shadow, movements. Using similar triangles and trigonometric functions, we managed to calculate the shadow, and using it to find its computation resources. Using three-dimensional geometry, we were able to find the movement of an object. Using determining statements, we were able to find the collision. We have also created a calculation depth, to modify the situation through multiple cases. **Overall, the range of computing resources to maintain virtual city of Hong Kong requires $[6.373416 \times 10^{12}, 5.023116 \times 10^{13}]$.**

Contents

I. SUMMARY	1
II. INTRODUCTION	3
III. DEFINING THE PROBLEM.....	3
3.1 QUESTION RESTATEMENT	3
3.2 ASSUMPTIONS.....	3
3.3 VARIABLES	5
IV. BUILDING BLOCKS	5
5.1 INTRODUCTION	5
5.2 ENTITY	5
5.3 TEXTURE	6
V. STORAGE RESOURCE.....	7
VI. COMPUTING RESOURCE.....	8
VII.	12
VIII. STRENGTHS	13
IX. WEAKNESSES.....	13
X. CONCLUSION.....	14
XI. APPENDIX.....	14

II. Introduction

Our model will portray a virtual Hong Kong in which its working mechanism is synchronized accordingly with the real-life Hong Kong. It contains the multiple aspects significant to life in this place like cars, people, buildings, etc., which can create a more realistic world overall. In the model we have also taken the motion of specific objects like water and cars in account to calculate the computing resource. After doing research about the Hong Kong geography, vehicles, population, landmarks, climate, etc., we have created a world which can actively model the real-time weather and the overall appearance of Hong Kong.

Before explaining the problem, we will begin first by defining what the problem is requiring.

III. Defining the Problem

3.1 Question restatement

This question requires us to:

1. Design a set of building blocks that can be used to build a virtual Hong Kong in the virtual world.
2. Estimate the computing and storage resources required to maintain such a virtual Hong Kong in real-time, and determine the time precision of synchronization. In this paper we will estimate the computations per second and the bytes of storage needed for the virtual world of Kong Hong. Given these objectives, we can begin laying out the assumptions and variables we need to account for and define.

3.2 Assumptions

In designing our model, we decided on several basic assumptions. Listed below are the assumptions and our justifications for them.

Assumption	Justification
Every building's story is 2.5 meters	The law requires that every habitable room shall have a height of at least 2.5m measured from floor to ceiling [1]. We are following the legal requirements of the Hong Kong government
Minor (defined as those which do not have a significant impact on the overall virtual model of Hong Kong) details like trash cans and street lights are not taken account into the model.	An excess number of objects that both do not have a meaningful effect with the experience of the virtual world and take up superfluous storage space can reduce unwanted errors and lagging in the world
Raindrops are spherical.	If raindrops were to be programmed as a combination of a cone and a half sphere, along with the fact that there would be a lot of rain drops during the stage, then it would take up too much storage

	space and computing resource due to the need to store the movement of every single rain drop. It would lag the world significantly and create problems for the visage quality
Clouds move at a constant velocity.	The need to store the movement of each cloud would again take up too much computing resource, since if the clouds' tracks follow that of the erratic wind, then the movements would be excess space to take up. In addition, them being precise and realistic would not have momentous value to the world
The length of day and night are the same	This would make the calculation easier by not taking into account how the lengths of the day and night vary according to the differing seasons, which will create more computing resources for other more important activities like the movement of cars and people
The water in streams, lakes, and oceans, do not move	Calculating the movement of the water moving realistically and its path would be a waste of the space it takes up in computing
The vehicles that are portrayed in the virtual world are sedans, trucks, nonmotorized vehicles (motorcycles and bicycles), airplanes (economy and helicopter), trains (subways and railroad car), and ships (yachts and ferries)	It would be impossible to take every kind of vehicle that exists in Hong Kong into account and decide the number for each kind. Therefore, we take the most common kind of car in Hong Kong, sedan, and add the truck alongside
All vehicles of the same name have the same dimensions, but can have different color	There are many types of models for even one car. Sedans are available in a range of sizes from small to compacts to mid-size and full-size [2], so if we were to take each kind of model and their dimensions into the calculation, it would take up huge amounts of computing resources that should be left for other models
One addition and subtraction, multiplication and division all count as one computation	Most modern CPUs come with multipliers, and multiplication must be faster than cyclic addition because of a different algorithm. The optimizations of different compilers can also affect the number of actual computations, similarly with division, so we make this assumption

3.3 Variables

Variable	Definition
r	The level of red of a texture
g	The level of green of a texture
b	The level of blue of a texture
a	The transparency level of a texture
n	The number of terms in the Taylor Series expansions of a trigonometric operation
T(n)	A function that tells the time complexity of an algorithm
fps	Frame per second, or the refresh rate of the virtual world
T _o	The total number of entities currently in the virtual world
T _{do}	The total number of moving objects currently in the virtual world

IV. Building Blocks

5.1 Introduction

Creating a virtual Hong Kong requires a lot of different shapes and different texture. This can be hard if we wanted to define the set of building blocks as bricks, windows, stairways.... There is a high possibility that the set is going to be too complicated or not be able to include. It is hard to keep it as simplest as possible and also to give players full creativity.

Our solution is to divide the whole set of building blocks into two different parts: the entity and texture. The main idea is to divide Hong Kong into a great number of regular geometric shapes. Then, for each shape in the virtual city, players can put on texture to make it look realistic. For example, if there is window where the players must create, then the player will use a very flat rectangular prism and put a texture that have very high transparency to make it look like a window. Combining the entities and textures can lead to a wide variety of objects in real life with less learning difficulties.

5.2 Entity

Entity is the part in our set of building blocks that will take up the shape of Hong Kong. We have defined 4 main types of basic entity based on their geometric shapes that can be able to include enough details. The entity will have numerous dimensions for many different uses (ex. Creating windows). Also, players can create their own blocks by inputting wanted dimensions (in a limited scale).

The basic entities are listed below.

1. Prism

- Rectangular Prism (length * width * height)
 - (5 m * 0.5 m * 5m), (1 m * 0.1 m * 1 m), (20 m * 20 m * 1 m), (10 m * 5 m * 0.1 m), etc.
 - Cube (side * side * side)
 - Basic Building Cube (0.1 m * 0.1 m * 0.1 m)
 - Triangular Prism / Slopes (base leg 1, base leg 2, height)
 - Cylinder (radius, height)
 - (0.3 m, 0.1 m), (0.1 m, 1 m), (0.3 m, 0.3 m), etc.
 - Other prisms
2. Pyramid
- Square Pyramid (side, height)
 - Tetrahedron (side)
 - Other Pyramids
3. Sphere
- Sphere (radius)
 - (0.2 m)
 - Semi-sphere (radius)
 - Other
4. Plane
- Sea Level
 - Signs
 - Patterns

5.3 Texture

We have created our texture by defining into 4 basic variables: (r, b, g, a). r, b, g defines the color of the texture, and a defines the transparency of an object.

Using this, some basic textures are listed below.

1. Visible Colors
2. Transparent
3. Water

Instead of colors, transparent, water, light, we have also created complex textures for some special types of objects:

Textures	Divisions	r	g	b	a
Transparent	Full Transparent	254	254	254	255
	Half Transparent	245	245	245	128
Visible Color	Red	255	0	0	0
	Orange	255	165	0	0
	Yellow	255	255	0	0
	Green	0	128	0	0
	saddle brown	139	69	19	0
	Blue	0	0	255	0
	purple	128	0	128	0
	White	255	255	255	0
	Brown	165	42	42	0
	Grey	128	128	128	0
	Black	0	0	0	0

Figure 1: The Table of Textures

For some more complex shapes, such as vehicles, we already included to our entity list (due to the complexity of its creation). Complex building blocks have their own textures.

The complex building block are the following below.

Complex Build	Divisions	Subdivisi	Entity	r	g	b	a
Vehicles	Big Vehicles		big vechicles	Visible colors	Visible colors	Visible colors	Visible colors
	Cars		cars	Visible colors	Visible colors	Visible colors	Visible colors
	Non-motorized vehicles		non-motorized vehicles	Visible colors	Visible colors	Visible colors	Visible colors
	airplanes		airplanes	Visible colors	Visible colors	Visible colors	Visible colors
	trains		trains	Visible colors	Visible colors	Visible colors	Visible colors
	sea vessels			Visible colors	Visible colors	Visible colors	Visible colors
Climate	Prepitation	rain	sphere	30	144	255	254
		water	planes	30	144	255	75
		snow	sphere	255	255	255	0
		clouds	cloud	245	245	245	50
		lightening	lightening	255	255	0	0
	celestial body	sun	sphere	255	255	0	0
		moon	sphere	245	245	245	0
		stars	sphere	245	245	245	0

Figure 2: The Table of Complex Building Blocks

V. Storage Resource

The storage resource of our virtual Hong Kong is a collection of the following: list of objects, entities, and textures. To better manifest the division of storage, we constructed structures for entities, textures, and objects.

Entities have two basic parameters, which are their dimensions and flags. Their dimension considers the spatial aspect - representing the maximum length, width, and height that they take, a rectangular prism. However, not every point inside the prism is part of the entity, or else every entity will look like rectangular prisms. A flag is a 3-dimensional Boolean list regarding the true or false that the input x, y, and z value in the dimension is part of the entity or void. Entities are decorated with textures while void can be interpreted as transparent air. Basically, flag gives the shape of entities when given the dimension. With given dimension and flag, every shape can be created, regarding those with curves and complex attributes.

On the other hand, textures are composed of the RGB values and transparency value. RGB is an indicator of colors, for red, green, blue are primary colors that can combine in various amounts to obtain any other color from the visible spectrum that the human eye can see [3]. It is the standard method of producing color images. Considering that some entities are transparent rather than opaque, such as water and clouds, we appended in a transparency factor 'alpha' so that transparent objects like raindrops will not be a blue sphere which players cannot see through. Afterall, the purpose of creating a virtual world is to mimic the presentiment of the actual world. RGB and alpha have a maximum value of 255 and minimum value of 0 because the integer 255 is 11111111 in binary, exactly 8 bits. RGB color mode is mainly used in light-emitting diodes with single-chip microcomputer as the core of operation. Data in the single-chip chip are stored and calculated in the form of binary. Most chips were 8 bits, meaning that the maximum capacity of information stored in each module was exactly 2^8 , or 256 (from 0 to 255).

The object structure, or the structure of building blocks, contain its coordinate, employed entity, and texture. Integers x, y, and z make up the coordinate tuple. The entity parameter takes entity structures while the texture parameter takes texture structures - structures inside structures, making the correlations between the building blocks and their entities and textures lucid.

To compute the total storage that our virtual Hong needs, we simply added the total bytes from the web of objects, entities, and textures. Integer values take up 4 bytes each as Boolean values take up 1 byte each.

In conclusion, the size of the Storage in bytes is the sum of Entity, texture, and object list, which is the number of entity * 2040 + 26364. Based on Hong Kong's statistics, we estimated the number of entities to be 3600798691 (Actual Number depends on the entity that the players will use). The estimated storage is 6841 GB of storage.

VI. Computing Resource

When it comes to calculating how many computations in one second, one decisive question is how we define one "computation." Modern chips calculate the sum, difference, product, and quotient of two numbers in the best, optimized way. For example, chips will not calculate multiplications $5 * 5$ by finding the sum of five 5s, for it takes up more space and time. Special techniques such as bitwise operations, sign magnitude notations, and Booth algorithm are utilized in different circumstances. Therefore, it is impossible to achieve what has the computer performed in each operation without knowing how different chips work, which are full of intricate relations and judgements. However, what we can find is the number of additions, subtractions, multiplications, and divisions that our virtual Hong Kong processes, and when we assume each basic operation as one "computation," we can estimate the range of the computing resource required.

In our virtual Hong Kong, we organized the needed computations into the following: the positions of moving objects, the directions of shadows produced by the sun, the lengths of shadows, and object collisions.

To begin, we expanded and converted some special operations into those that we count as one computation.

TRIGONOMETRY

In our virtual world, trigonometry is a basic operation and heavily used in myriad situations to calculate values like distance and angles. To obtain the number of computations in sine and cosine, we expand the sine and cosine into an infinite sum of numbers, known as the Taylor Series[4]. The more terms used to calculate the sum, the more accurate the result will be. But, with more terms come with more computations - a quadratic growth. 'N' is the variable defining the number of terms used to calculate sine and cosine. Therefore, the number of computations in one trigonometric operation can be expressed by: $3n^2 + 2n - 1$.

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots \quad -\infty < x < \infty$$

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \dots \quad -\infty < x < \infty$$

$$\sin x = \sum_{i=0}^n -1^i \times \frac{x^{2i+1}}{(2i+1)!}$$

$$\cos x = \sum_{i=0}^n -1^i \times \frac{x^{2i}}{(2i)!}$$

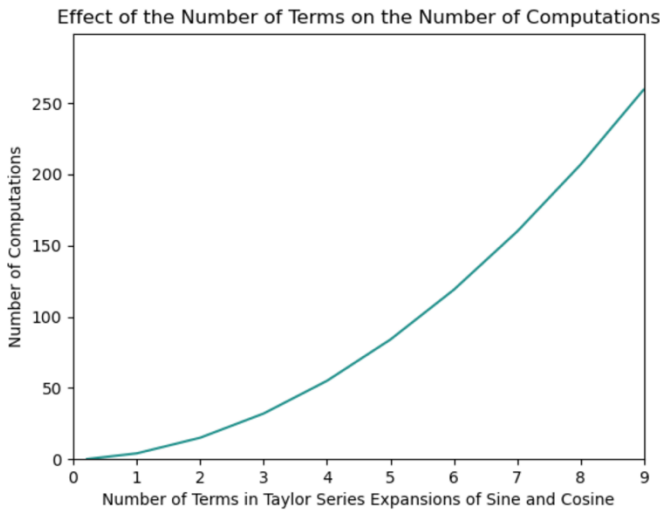


Figure 3: The Growth of the Computing Resources with the accuracy of $\sin(x)$ and $\cos(x)$

SQUARE ROOTS

Square root is a less utilized operation, yet we still need to find the number of computations, for it is employed in finding the shadow lengths. We use the iterative method to get the root of a

number, which was invented by the ancient Babylonians (1500 BC) and the Greeks (100 AD) long before Newton invented his general procedure[5]. The method works as the following:

1. Make an initial guess of the root.
2. Improve the guess.
3. Iterate until convergence.

Similarly, as the expansion of trigonometry, the more iterations, the more computations there are. We choose to use its time complexity $T(n^{0.5})$ to represent the number of iterations so that the number of computations depends on the number being asked to find the root of, which we can get a range from, explained later in GET_SHADOW_LENGTH.

GET_ANGLE

GET_ANGLE is a function that returns the angle between the x-axis and the line formed by connecting two points. It only uses the x and y coordinate of two points and works on a 2-dimensional Cartesian plane. This function is utilized to calculate the angle that the shadow projects on the ground by any light source.

Total Computations: $3n^2 + 2n + 2$

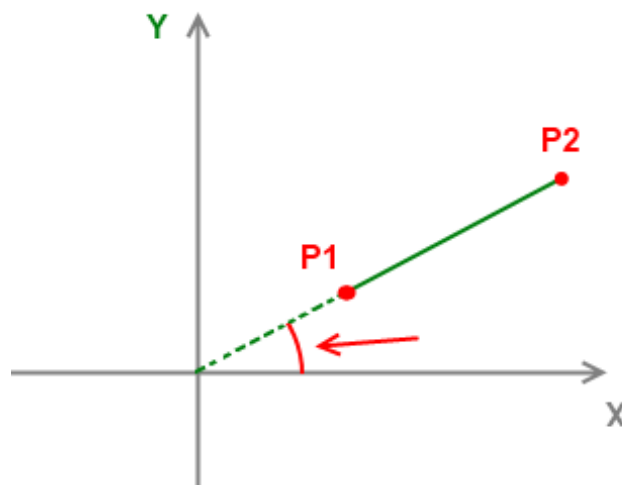


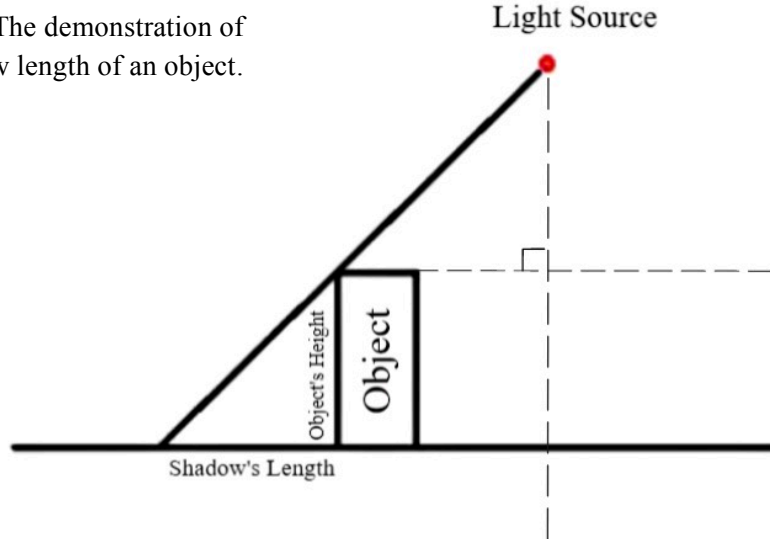
Figure 4: The Angle that shadow projects on the ground

GET_SHADOW_LENGTH

This function is solely utilized to return the length of shadows casted by a light source by inputting the coordinates of the light source and the object and the height of the object. We do not know the exact number of computations when invoking the square root function, but we know the range of the number that will be computed to find its square root, which is $\{0 \leq x \leq 4562.5\}$, achieved by using the dimension of Hong Kong.

Total Computations: $8 \leq \text{total computation} \leq 211$

Figure 5: The demonstration of the shadow length of an object.



JUDGE_COLLISION

Object collision is detected in this Boolean-return function with a given point and plane. The function merely determines whether the point belongs to the plane or not, and there are no true computations that needs to perform here.

Total Computations: 0 (but, there are 10 determine statements)

GET_POS

GET_POS returns the new coordinate that a moving object will be in the next frame with given velocity. Velocity is expressed by magnitude and direction. Magnitude is an integer value representing the object's speed. Direction is a tuple in length of two signifying angles y and z .

Total Computations: $15n^2 + 10n + 4$

The total computation resource used is the sum of the calculations in each function. Some objects will invoke certain functions while others do not. Here, we note the total number of dynamic (moving) entities and total number of entities as T_{de} and T_e respectively. Moreover, the FPS (frame per second) value will impact onto the number of computations per second. The FPS of our virtual Hong Kong specifies how many times the screen is refreshed to produce a new frame. The more frames there are in one second, the more computations the computer will perform, forming a direct

variation. At last, the estimated range of total computations per second with all the functions and objects considered is:

$$[\text{fps} * (T_o * (3n^2 + 2n + 10) + T_{do} * (15n^2 + 10n + 4)), \text{fps} * (T_o * (3n^2 + 2n + 213) + T_{do} * (15n^2 + 10n + 4))]$$

We have estimated that T_o is 3600800000 (rounded), T_{do} is the half of T_o . When $n = 1$ and FPS is 60, the estimated range of computation resources is $[6.373416 \cdot 10^{12}, 5.023116 \cdot 10^{13}]$.

VII. Sensibility Analysis

In our model of computing resources, we have two variables FPS and n . Using one-at-a-time technique, we compared the results in changing the variables of FPS and n .

When we have changed n (FPS = 60, $T_o = 3600800000$, $T_{do} = 1800400000$), the results are the following.

n	computing resources per second range
1	[6373416*10 ⁶ , 5023116*10 ⁷]
2	[14691264*10 ⁶ , 58549008*10 ⁶]
3	[2754612*10 ⁷ , 71403864*10 ⁶]
4	[44937984*10 ⁶ , 88795728*10 ⁶]
5	[66866856*10 ⁶ , 1107246*10 ⁸]

Figure 6: The Relationship between n and computing resources

Changing n by 1 creates dramatic results. The Greatest Change is around $2.2 \cdot 10^{13}$.

When we have changed FPS ($n = 1$, $T_o = 3600800000$, $T_{do} = 1800400000$), the results are the following.

FPS	Computing Resources per second Range
40	[4248944*10 ⁶ , 3348744*10 ⁷]
50	[531118*10 ⁷ , 418593*10 ⁸]
60	[6373416*10 ⁶ , 5023116*10 ⁷]
70	[7435652*10 ⁶ , 5860302*10 ⁷]
80	[8497888*10 ⁶ , 6697488*10 ⁷]

Figure 7: The Relationship between FPS and computing resources

Changing FPS by 10 creates less dramatic results; however, the change in results is still very high. The Greatest Change is around 10^{12} .

The changes are huge because the T_o is too large (and it results in a large change in T_{do}). The computing resources are much higher in the actual situation as the players use more entity. When changing in the variables FPS and n , the computing resources needed will need huge upgrades.

VIII. Strengths

5.1 Before we even started on doing the model, our team did numerous research about the specifics of Hong Kong government and the overall “template” in which we would make the virtual world out of. The dimensions in which we wrote the models to be in were made according to the federal requirements and are therefore realistic and fitting. The statistics and data were both extremely important in the final result, and our team made sure to do thorough investigation in multiple concepts in order to ensure the credibility of the model.

5.2 Hong Kong is a complex place with unique characteristics and geography, so it was required to consider the problem in multiple aspects that define Hong Kong as it is. Our team included many different features in the world, enriching the model furthermore, providing a better experience.

IX. Weaknesses

6.1 Though the existing numbers coincide with real life, our models may not be the most abundant. For example, the vehicles that we programmed to be on the roads of Hong Kong only included the sedan of one kind of model and the truck. Though we have included many aspects, they are only representations of the many other similar objects. Therefore, there were not a lot of variations for the different models, which may have prevented the virtual world from becoming more realistic, but on the whole, the model still presents a believable and credible portrayal of Hong Kong.

6.2 Our second weakness is that some of the illustrations in the virtual world may not be extremely true-to-life, since we only had a limited number of building blocks in which we made them as geometric 3D models. It is challenging to recreate every single thing in Hong Kong as it is using just the blocks which we created. For example, a tree would be a cylinder with a sphere situated above it painted with the leaf complex block, which is not a concise representation of the real-life tree. However, even though the depicted images may not be 100% naturalistic, they are still extremely effective in delivering the supposed imagery on the whole.

X. Conclusion

In conclusion, after research of Hong Kong's geography, we divided the parts that we needed to calculate the storage resource of into vehicles, buildings, people, weather (synchronized with real-life Hong Kong), and the things of nature like trees and the fluidity of water.

In order to ensure that the virtual world would function favorably and without significant lagging, our team decided to use the least amount of building blocks which still have the ability to construct identifiable objects, at last creating the 3D geometric "blocks" on four main types of prisms, two main pyramids, a sphere and a semi-sphere, and the different types of planes for signs and other patterns. By doing so, we have fulfilled the fundamental requirements.

After calculation and analysis, we got the results of the storage resource being 6841 GB of storage and the computing resource being between $6.373416 * 10^{12}$ and $5.023116 * 10^{12}$, creating the desired effects of a virtual Hong Kong world!

XI. Appendix

Reference:

- [1] [990in3.pdf \(legco.gov.hk\)](#)
- [2] <https://www.caranddriver.com/shopping-advice/g26100588/car-types/>
- [3] <https://www.digitalcitizen.life/rgb-rgb-lighting/>
- [4] <https://www.sciencedirect.com/topics/engineering/taylor-series>
- [5] <https://blogs.sas.com/content/iml/2016/05/16/babylonian-square-roots.html#:~:text=The%20Babylonian%20square-root%20algorithm%20The%20iterative%20method%20is,AD%29%20long%20before%20Newton%20invented%20his%20general%20procedure.>